

Requirements & Fitness-for-Purpose Analysis

Spring Web Service Sandbox

Pre-Planning Consultation by Metis Agent

Project: spring-web-service-sandbox

Stack: Java 25 · Spring Boot 4.0.3 · Hexagonal Architecture

Date: 9 March 2026

1. Purpose Analysis

This project is clearly a **learning sandbox and portfolio piece**. It demonstrates: hexagonal architecture, async messaging, observability, Testcontainers, and clean layering.

It currently lives in a middle ground where it is too simple to be a compelling reference implementation but too structured to be a throwaway experiment. The hexagonal architecture wraps a domain so thin that it risks looking like architecture for architecture's sake. A reviewer would notice immediately that the domain layer has zero business logic — `BookService` is pure delegation.

2. Missing Questions

1. **Who is the intended audience?** A junior developer learning Spring Boot sees different value than a senior architect evaluating hexagonal patterns.
2. **What does “done” look like?** No backlog, no roadmap, no TODO list.
3. **What should this project NOT demonstrate?** Without explicit exclusions, every new Spring Boot feature becomes a candidate.
4. **Is the ISBN format intentionally restrictive?** The regex accepts only one hyphenation pattern.
5. **What happens to a 10K-record import if RabbitMQ goes down mid-stream?** Partial failure semantics are undefined.
6. **Is duplicate ISBN detection intended?** No unique constraint exists.

3. Undefined Guardrails

Guardrail	Current State	Suggested Definition
API versioning	No version prefix on <code>/books</code>	Document as out of scope for sandbox

Data volume limits	<code>GET /books</code> returns every row unbounded	Add pagination or document limitation
Rate limiting / auth	All endpoints wide open	Add “Security” section to README
CSV row count cap	Only 10MB multipart limit	Consider row-count cap or document behaviour
Actuator security	All endpoints publicly accessible including <code>/actuator/env</code>	Restrict to local profile or document as dev-only

4. Scope Risks

Observability Scope Creep

The project already has Prometheus, Grafana, and a custom dashboard. The natural next step is tracing (Zipkin/Jaeger), structured logging, more dashboards. Each adds complexity without advancing the core demonstration.

Prevention: Define which observability pillars are in scope (metrics: yes, tracing: no).

CRUD Completion Pressure

The API has Create and Read but no Update or Delete. Adding these is tempting but each introduces questions (partial updates via PATCH? soft delete? cascading?).

Prevention: Define specific acceptance criteria or document the exclusion.

Multi-Entity Domain Expansion

Adding Author, Publisher, or Categories would make the domain richer but transforms this from a sandbox into a real application.

Prevention: Decide whether the project demonstrates architecture (keep simple) or domain modeling (add richness).

5. Unvalidated Assumptions

Assumption	Risk	Validation Needed
Java 25 is LTS	Running non-GA or early-LTS JDK creates friction	Verify GA status and vendor commitments

Spring Boot 4.0.3 is stable	Ecosystem gaps (Lombok, springdoc compatibility)	Confirm build passes and Swagger UI renders
<code>host.docker.internal</code> works everywhere	Fails on native Linux Docker	Document platform requirement
Jackson serialization works across services	Would break in a microservice split	Fine for intra-service; document limitation
Domain is framework-free	<code>@Service</code> annotation violates this	Decide: fix or acknowledge compromise

6. Missing Acceptance Criteria

Bulk Import Feedback

The import endpoint returns `202 Accepted` with no body. No way for the caller to know outcomes.

Criterion: Response should include `{ "enqueued": N, "skipped": M }` at minimum.

Error Response Consistency

`GlobalExceptionHandler` returns `Map<String, String>` for some errors, but other exceptions produce Spring Boot defaults.

Criterion: All 4xx/5xx responses should follow a consistent error envelope.

Validation Authority

Validation exists only at the web adapter. Messages via RabbitMQ bypass all validation entirely.

Criterion: Define where validation is authoritative — at the domain boundary or at each adapter.

Import Idempotency

Same CSV uploaded twice creates all records again.

Criterion: Define whether duplicates are acceptable or ISBN uniqueness should be enforced.

7. Edge Cases

Scenario	Current Behaviour	Risk
CSV with missing columns	<code>IllegalArgumentException</code> , unhandled → 500	High
Empty CSV file	202 with no feedback	Low
Very long field values	No <code>@Size</code> constraints → <code>DataIntegrityViolationException</code>	Medium
Non-numeric ID (<code>GET /books/abc</code>)	<code>MethodArgumentTypeMismatchException</code> , unhandled	Medium
RabbitMQ poison message	Infinite redelivery, no DLQ	High
10K individual INSERTs	No batching, no transaction grouping	Medium (performance)

8. Technology Choices

Choice	Assessment
Java 25 / Spring Boot 4.0.3	Cutting-edge, forward-thinking but creates adoption friction. Needs <code>.sdkmanrc</code> .
Hexagonal Architecture	Well-structured but under-utilized. Domain has no rules to justify it.
Lombok	Pragmatic but controversial in a portfolio piece. Some teams reject it.
MySQL 8.4	Solid. Versions match between docker-compose and Testcontainers.
RabbitMQ 4	Good match for queue-based load leveling pattern.
Flyway	Appropriate. Single migration file is clean.
springdoc-openapi 3.0.2	Needs Boot 4.x compatibility verification.

9. Domain Model Adequacy

The `Book` record with four string fields and an ID is a data carrier, not a domain model. There is no behavior, no invariants, no business rules. This is the single biggest gap if the project aims to demonstrate hexagonal architecture.

What Could Be Added Without Scope Explosion

- ISBN as a value object (self-validating, with check digit verification)
- Book status (Available, Checked Out, Reserved) with state transition rules
- Publication year with invariant (cannot be in the future)
- `BookAlreadyExistsException` for duplicate ISBN
- Domain events (`BookCreated` , `BookImported`)

10. Recommendations (Prioritized)

#	Action	Impact	Effort
1	Add one meaningful domain rule (ISBN uniqueness with domain exception)	HIGH	LOW
2	Add pagination to <code>GET /books</code>	HIGH	LOW
3	Add dead-letter queue for RabbitMQ	HIGH	LOW
4	Return import statistics from CSV endpoint	MEDIUM	LOW
5	Add <code>@Size</code> constraints matching database column widths	MEDIUM	LOW
6	Move validation into domain layer	MEDIUM	MEDIUM
7	Add UPDATE and DELETE endpoints	MEDIUM	MEDIUM
8	Handle non-happy-path exceptions in <code>GlobalExceptionHandler</code>	MEDIUM	LOW
9	Add <code>.sdkmanrc</code> file	LOW	TRIVIAL
10	Document the <code>@Service</code> trade-off or fix it	LOW	TRIVIAL

Summary

This is a well-organized sandbox that demonstrates good engineering instincts. The primary gap is that the architecture is more sophisticated than the domain it protects, which undermines the demonstration value.

Adding even one meaningful business rule would transform the project from “I can structure code” to “I understand why we structure code this way.”

The secondary gap is defensive coding: the happy path works cleanly, but edge cases reveal that production-readiness thinking has not been applied uniformly.